

# Лекция 7: вывод типов

Функциональное программирование на Haskell

---

Алексей Романов

26 сентября 2026 г.

МИЭТ

## Вывод типов в других языках

- Во многих ООП-языках сейчас есть вывод типов локальных переменных (иногда и возвращаемого типа):

## Вывод типов в других языках

- Во многих ООП-языках сейчас есть вывод типов локальных переменных (иногда и возвращаемого типа):
  - `auto` в C++11
  - `var` в C# 3.0 и Java 10
  - Отсутствие явного типа в Kotlin, Scala

## Вывод типов в других языках

- Во многих ООП-языках сейчас есть вывод типов локальных переменных (иногда и возвращаемого типа):
  - `auto` в C++11
  - `var` в C# 3.0 и Java 10
  - Отсутствие явного типа в Kotlin, Scala
- Устроен в них всех похоже:

## Вывод типов в других языках

- Во многих ООП-языках сейчас есть вывод типов локальных переменных (иногда и возвращаемого типа):
  - `auto` в C++11
  - `var` в C# 3.0 и Java 10
  - Отсутствие явного типа в Kotlin, Scala
- Устроен в них всех похоже:
  - Тип аргументов функций задаётся явно.

## Вывод типов в других языках

- Во многих ООП-языках сейчас есть вывод типов локальных переменных (иногда и возвращаемого типа):
  - `auto` в C++11
  - `var` в C# 3.0 и Java 10
  - Отсутствие явного типа в Kotlin, Scala
- Устроен в них всех похоже:
  - Тип аргументов функций задаётся явно.
  - Дальше типы протягиваются сверху вниз.

## Вывод типов в других языках

- Во многих ООП-языках сейчас есть вывод типов локальных переменных (иногда и возвращаемого типа):
  - `auto` в C++11
  - `var` в C# 3.0 и Java 10
  - Отсутствие явного типа в Kotlin, Scala
- Устроен в них всех похоже:
  - Тип аргументов функций задаётся явно.
  - Дальше типы протягиваются сверху вниз.
  - Тип локальных переменных/полей = тип инициализатора (если нет явного).

## Вывод типов в других языках

- Во многих ООП-языках сейчас есть вывод типов локальных переменных (иногда и возвращаемого типа):
  - `auto` в C++11
  - `var` в C# 3.0 и Java 10
  - Отсутствие явного типа в Kotlin, Scala
- Устроен в них всех похоже:
  - Тип аргументов функций задаётся явно.
  - Дальше типы протягиваются сверху вниз.
  - Тип локальных переменных/полей = тип инициализатора (если нет явного).
  - Возвращаемый тип = общий тип для всех `return`.

## Вывод типов в других языках

- Во многих ООП-языках сейчас есть вывод типов локальных переменных (иногда и возвращаемого типа):
  - `auto` в C++11
  - `var` в C# 3.0 и Java 10
  - Отсутствие явного типа в Kotlin, Scala
- Устроен в них всех похоже:
  - Тип аргументов функций задаётся явно.
  - Дальше типы протягиваются сверху вниз.
  - Тип локальных переменных/полей = тип инициализатора (если нет явного).
  - Возвращаемый тип = общий тип для всех `return`.
- Отдельный вопрос: вывод параметров шаблонов (в C++)/генериков (в других языках).

## Вывод типов в других языках

- Во многих ООП-языках сейчас есть вывод типов локальных переменных (иногда и возвращаемого типа):
  - `auto` в C++11
  - `var` в C# 3.0 и Java 10
  - Отсутствие явного типа в Kotlin, Scala
- Устроен в них всех похоже:
  - Тип аргументов функций задаётся явно.
  - Дальше типы протягиваются сверху вниз.
  - Тип локальных переменных/полей = тип инициализатора (если нет явного).
  - Возвращаемый тип = общий тип для всех `return`.
- Отдельный вопрос: вывод параметров шаблонов (в C++)/генериков (в других языках).
- Он сложнее, но появился раньше.

## Вывод типов в системе Хиндли — Милнера

- В основе системы типов языка Haskell лежит система Хиндли — Милнера.
- Она изначально создана для вывода типов.

## Вывод типов в системе Хиндли — Милнера

- В основе системы типов языка Haskell лежит система Хиндли — Милнера.
- Она изначально создана для вывода типов.
- Даже без указания типов параметров.

## Вывод типов в системе Хиндли — Милнера

- В основе системы типов языка Haskell лежит система Хиндли — Милнера.
- Она изначально создана для вывода типов.
- Даже без указания типов параметров.
- Плюсы:

## Вывод типов в системе Хиндли — Милнера

- В основе системы типов языка Haskell лежит система Хиндли — Милнера.
- Она изначально создана для вывода типов.
- Даже без указания типов параметров.
- Плюсы:
  - Для любого данного выражения есть наиболее общий тип (если есть хоть какой-то).

## Вывод типов в системе Хиндли — Милнера

- В основе системы типов языка Haskell лежит система Хиндли — Милнера.
- Она изначально создана для вывода типов.
- Даже без указания типов параметров.
- Плюсы:
  - Для любого данного выражения есть наиболее общий тип (если есть хоть какой-то).
  - Есть эффективные (для реальных программ) алгоритмы их нахождения.

## Вывод типов в системе Хиндли — Милнера

- В основе системы типов языка Haskell лежит система Хиндли — Милнера.
- Она изначально создана для вывода типов.
- Даже без указания типов параметров.
- Плюсы:
  - Для любого данного выражения есть наиболее общий тип (если есть хоть какой-то).
  - Есть эффективные (для реальных программ) алгоритмы их нахождения.
- Не все расширения Haskell их сохраняют (но это один из критериев оценки расширений).

## Вывод типов в системе Хиндли — Милнера

- В основе системы типов языка Haskell лежит система Хиндли — Милнера.
- Она изначально создана для вывода типов.
- Даже без указания типов параметров.
- Плюсы:
  - Для любого данного выражения есть наиболее общий тип (если есть хоть какой-то).
  - Есть эффективные (для реальных программ) алгоритмы их нахождения.
- Не все расширения Haskell их сохраняют (но это один из критериев оценки расширений).
- Минусы:

## Вывод типов в системе Хиндли — Милнера

- В основе системы типов языка Haskell лежит система Хиндли — Милнера.
- Она изначально создана для вывода типов.
- Даже без указания типов параметров.
- Плюсы:
  - Для любого данного выражения есть наиболее общий тип (если есть хоть какой-то).
  - Есть эффективные (для реальных программ) алгоритмы их нахождения.
- Не все расширения Haskell их сохраняют (но это один из критериев оценки расширений).
- Минусы:
  - Сложнее понять, как работает.

## Вывод типов в системе Хиндли — Милнера

- В основе системы типов языка Haskell лежит система Хиндли — Милнера.
- Она изначально создана для вывода типов.
- Даже без указания типов параметров.
- Плюсы:
  - Для любого данного выражения есть наиболее общий тип (если есть хоть какой-то).
  - Есть эффективные (для реальных программ) алгоритмы их нахождения.
- Не все расширения Haskell их сохраняют (но это один из критериев оценки расширений).
- Минусы:
  - Сложнее понять, как работает.
  - Больше «дальнодействия».

## Вывод типов в системе Хиндли — Милнера

- В основе системы типов языка Haskell лежит система Хиндли — Милнера.
  - Она изначально создана для вывода типов.
  - Даже без указания типов параметров.
  - Плюсы:
    - Для любого данного выражения есть наиболее общий тип (если есть хоть какой-то).
    - Есть эффективные (для реальных программ) алгоритмы их нахождения.
  - Не все расширения Haskell их сохраняют (но это один из критериев оценки расширений).
  - Минусы:
    - Сложнее понять, как работает.
    - Больше «дальнодействия».
  - Поэтому понимание работы иногда необходимо.
- 3/31

# Постановка задачи

- Даны:
  - Набор типов и конструкторов типов: `Int`, `Bool`, `[]` и т.д.
  - Среди них особую роль играет конструктор функций `->`.
  - Набор известных именованных функций (или констант) с их типами.
  - Выражение, тип которого мы хотим найти (обычно определение функции).

# Постановка задачи

- Даны:
  - Набор типов и конструкторов типов: `Int`, `Bool`, `[]` и т.д.
  - Среди них особую роль играет конструктор функций `->`.
  - Набор известных именованных функций (или констант) с их типами.
  - Выражение, тип которого мы хотим найти (обычно определение функции).
- Нужно:
  - Определить наиболее общий тип для этого выражения.

# Постановка задачи

- Даны:
  - Набор типов и конструкторов типов: `Int`, `Bool`, `[]` и т.д.
  - Среди них особую роль играет конструктор функций `->`.
  - Набор известных именованных функций (или констант) с их типами.
  - Выражение, тип которого мы хотим найти (обычно определение функции).
- Нужно:
  - Определить наиболее общий тип для этого выражения.
  - Или найти объяснение, почему ему нельзя дать никакой тип.

- Сравните `let x = e1 in e2` и `(\x -> e2) e1`.

- Сравните `let x = e1 in e2` и `(\x -> e2) e1`.
- Для нерекурсивного `let`, если оба выражения типизируются, их значения одинаковы.

- Сравните `let x = e1 in e2` и `(\x -> e2) e1`.
- Для нерекурсивного `let`, если оба выражения типизируются, их значения одинаковы.
- Но есть разница при типизации:

- Сравните `let x = e1 in e2` и `(\x -> e2) e1`.
- Для нерекурсивного `let`, если оба выражения типизируются, их значения одинаковы.
- Но есть разница при типизации: `x` в лямбде всегда мономорфна, а в `let` может быть полиморфной.

- Сравните `let x = e1 in e2` и `(\x -> e2) e1`.
- Для нерекурсивного `let`, если оба выражения типизируются, их значения одинаковы.
- Но есть разница при типизации: `x` в лямбде всегда мономорфна, а в `let` может быть полиморфной.
- Поэтому в нетипизированном и просто типизированном  $\lambda$ -исчислении `let` обычно не вводится.

## Виды выражений и типов

- В нашей системе 5 видов выражений:

## Виды выражений и типов

- В нашей системе 5 видов выражений:
- Названия переменных (в том числе функций)  $x$
- Применение функции к аргументу
- Лямбда-выражения
- `let`-выражения
- `case`-выражения

## Виды выражений и типов

- В нашей системе 5 видов выражений:
- Названия переменных (в том числе функций)  $x$
- Применение функции к аргументу
- Лямбда-выражения
- **let**-выражения
- **case**-выражения
- Типы  $t$ :

## Виды выражений и типов

- В нашей системе 5 видов выражений:
- Названия переменных (в том числе функций)  $x$
- Применение функции к аргументу
- Лямбда-выражения
- **let**-выражения
- **case**-выражения
- Типы  $t$ :
- Базовые типы **Int**, ...
- Переменные типов (греческие буквы  $\alpha, \beta, \dots$ )
- Тип функций  $t_1 \rightarrow t_2$
- Применение конструктора типов: **Maybe**  $t$ ,  $[t]$ ,  $(t_1, t_2)$

# Виды выражений и типов

- В нашей системе 5 видов выражений:
- Названия переменных (в том числе функций)  $x$
- Применение функции к аргументу
- Лямбда-выражения
- **let**-выражения
- **case**-выражения
- Типы  $t$ :
- Базовые типы **Int**, ...
- Переменные типов (греческие буквы  $\alpha, \beta, \dots$ )
- Тип функций  $t_1 \rightarrow t_2$
- Применение конструктора типов: **Maybe**  $t$ ,  $[t]$ ,  $(t_1, t_2)$
- Полиморфные типы:  $\forall \alpha_1 \dots \alpha_k t$

# Виды выражений и типов

- В нашей системе 5 видов выражений:
- Названия переменных (в том числе функций)  $x$
- Применение функции к аргументу
- Лямбда-выражения
- **let**-выражения
- **case**-выражения
- Типы  $t$ :
- Базовые типы **Int**, ...
- Переменные типов (греческие буквы  $\alpha, \beta, \dots$ )
- Тип функций  $t_1 \rightarrow t_2$
- Применение конструктора типов: `Maybe  $t$` , `[ $t$ ]`, `( $t_1, t_2$ )`
- Полиморфные типы:  $\forall \alpha_1 \dots \alpha_k t$
- $\forall$  только на внешнем уровне, не под  $\rightarrow$ !

# Унификация

- Нам понадобятся понятия подстановки и унификации. В общем случае:

# Унификация

- Нам понадобятся понятия подстановки и унификации. В общем случае:
- Пусть даны два выражения (терма) какого-то формального языка (или несколько пар).
- Они могут содержать переменные.

# Унификация

- Нам понадобятся понятия подстановки и унификации. В общем случае:
- Пусть даны два выражения (терма) какого-то формального языка (или несколько пар).
- Они могут содержать переменные.
- Подстановка переменных сопоставляет некоторым переменным выражения (которые тоже могут содержать переменные).

# Унификация

- Нам понадобятся понятия подстановки и унификации. В общем случае:
- Пусть даны два выражения (терма) какого-то формального языка (или несколько пар).
- Они могут содержать переменные.
- Подстановка переменных сопоставляет некоторым переменным выражения (которые тоже могут содержать переменные).
- Мы хотим знать, можно ли сделать такую подстановку, чтобы термы стали одинаковыми.

# Унификация

- Нам понадобятся понятия подстановки и унификации. В общем случае:
- Пусть даны два выражения (терма) какого-то формального языка (или несколько пар).
- Они могут содержать переменные.
- Подстановка переменных сопоставляет некоторым переменным выражения (которые тоже могут содержать переменные).
- Мы хотим знать, можно ли сделать такую подстановку, чтобы термы стали одинаковыми.
- Это достаточно широко применимое понятие.

# Унификация

- Нам понадобятся понятия подстановки и унификации. В общем случае:
- Пусть даны два выражения (терма) какого-то формального языка (или несколько пар).
- Они могут содержать переменные.
- Подстановка переменных сопоставляет некоторым переменным выражения (которые тоже могут содержать переменные).
- Мы хотим знать, можно ли сделать такую подстановку, чтобы термы стали одинаковыми.
- Это достаточно широко применимое понятие.
- Разрешимость задачи унификации и свойства решений зависят от структуры языка.

# Унификация типов

- Для вывода типов в системе Хиндли — Милнера случай один из самых простых: унификация первого порядка.
- Подстановки  $\sigma$  сопоставляют переменным типов  $\alpha, \beta, \dots$  *мономорфные* типы (без  $\forall$ ). Например,

$$\sigma = \{\alpha \mapsto \text{Int}, \beta \mapsto [\gamma]\}$$

- Подстановку можно применить к типу:

$$\sigma(\text{Maybe } \beta) = \text{Maybe } [\gamma]$$

- Композиция подстановок — подстановка.
- Мономорфные типы равны, если они совпадают синтаксически.

# Унификация типов

- Для вывода типов в системе Хиндли — Милнера случай один из самых простых: унификация первого порядка.
- Подстановки  $\sigma$  сопоставляют переменным типов  $\alpha, \beta, \dots$  *мономорфные* типы (без  $\forall$ ). Например,

$$\sigma = \{\alpha \mapsto \text{Int}, \beta \mapsto [\gamma]\}$$

- Подстановку можно применить к типу:

$$\sigma(\text{Maybe } \beta) = \text{Maybe } [\gamma]$$

- Композиция подстановок — подстановка.
- Мономорфные типы равны, если они совпадают синтаксически.
  - Т.е. синонимы типов должны быть уже раскрыты!

# Унификация типов

- Тип  $(\alpha, \alpha)$  более общий, чем  $(\text{Int}, \text{Int})$ , так как  $\exists \sigma \sigma((\alpha, \alpha)) = (\text{Int}, \text{Int})$ .
- Второй — частный случай первого.

# Унификация типов

- Тип  $(\alpha, \alpha)$  более общий, чем  $(\text{Int}, \text{Int})$ , так как  $\exists \sigma \sigma((\alpha, \alpha)) = (\text{Int}, \text{Int})$ .
- Второй — частный случай первого.
- $(\alpha, \beta)$  более общий, чем  $(\alpha, \alpha)$ .

# Унификация типов

- Тип  $(\alpha, \alpha)$  более общий, чем  $(\text{Int}, \text{Int})$ , так как  $\exists \sigma \sigma((\alpha, \alpha)) = (\text{Int}, \text{Int})$ .
- Второй — частный случай первого.
- $(\alpha, \beta)$  более общий, чем  $(\alpha, \alpha)$ .
- Вопрос: когда два типа более общи друг друга?

# Унификация типов

- Тип  $(\alpha, \alpha)$  более общий, чем  $(\text{Int}, \text{Int})$ , так как  $\exists \sigma \sigma((\alpha, \alpha)) = (\text{Int}, \text{Int})$ .
- Второй — частный случай первого.
- $(\alpha, \beta)$  более общий, чем  $(\alpha, \alpha)$ .
- Вопрос: когда два типа более общи друг друга?
- Когда они отличаются только названиями переменных.

# Унификация типов

- Тип  $(\alpha, \alpha)$  более общий, чем  $(\text{Int}, \text{Int})$ , так как  $\exists \sigma \sigma((\alpha, \alpha)) = (\text{Int}, \text{Int})$ .
- Второй — частный случай первого.
- $(\alpha, \beta)$  более общий, чем  $(\alpha, \alpha)$ .
- Вопрос: когда два типа более общи друг друга?
- Когда они отличаются только названиями переменных.
- Можем рассмотреть задачу унификации  $(\alpha, \beta) = (\beta, \text{Int})$ .

# Унификация типов

- Тип  $(\alpha, \alpha)$  более общий, чем  $(\text{Int}, \text{Int})$ , так как  $\exists \sigma \sigma((\alpha, \alpha)) = (\text{Int}, \text{Int})$ .
- Второй — частный случай первого.
- $(\alpha, \beta)$  более общий, чем  $(\alpha, \alpha)$ .
- Вопрос: когда два типа более общи друг друга?
- Когда они отличаются только названиями переменных.
- Можем рассмотреть задачу унификации  $(\alpha, \beta) = (\beta, \text{Int})$ .
- $\{\alpha \mapsto \text{Int}, \beta \mapsto \text{Int}\}$  это одно из её решений.

# Унификация типов

- Тип  $(\alpha, \alpha)$  более общий, чем  $(\text{Int}, \text{Int})$ , так как  $\exists \sigma \sigma((\alpha, \alpha)) = (\text{Int}, \text{Int})$ .
- Второй — частный случай первого.
- $(\alpha, \beta)$  более общий, чем  $(\alpha, \alpha)$ .
- Вопрос: когда два типа более общи друг друга?
- Когда они отличаются только названиями переменных.
- Можем рассмотреть задачу унификации  $(\alpha, \beta) = (\beta, \text{Int})$ .
- $\{\alpha \mapsto \text{Int}, \beta \mapsto \text{Int}\}$  это одно из её решений.
- Наиболее общее (все остальные — его частные случаи).

# Алгоритм унификации

- Вообще, у каждой задачи унификации типов есть наиболее общее решение (или нет вообще).
- Оно находится следующим алгоритмом:

# Алгоритм унификации

- Вообще, у каждой задачи унификации типов есть наиболее общее решение (или нет вообще).
- Оно находится следующим алгоритмом:
- На каждом шаге есть система уравнений. Выбираем любое из них и упрощаем.
- Закончим, когда не останется упрощаемых уравнений (оставшиеся опишут подстановку).
- Правило упрощения зависит от вида рассмотренного уравнения:

## Алгоритм унификации: правила (1)

- $C t_1 \dots t_k = C s_1 \dots s_k$  (одинаковые конструкторы):

## Алгоритм унификации: правила (1)

- $C t_1 \dots t_k = C s_1 \dots s_k$  (одинаковые конструкторы): заменяется на  $t_1 = s_1, \dots, t_k = s_k$ .

## Алгоритм унификации: правила (1)

- $C\ t_1 \dots t_k = C\ s_1 \dots s_k$  (одинаковые конструкторы): заменяется на  $t_1 = s_1, \dots, t_k = s_k$ .
  - $\text{Maybe } \alpha = \text{Maybe } [\text{Int}]$  даёт  $\alpha = [\text{Int}]$ .
  - $\alpha \rightarrow \beta = \text{Int} \rightarrow \text{Bool}$  даёт  $\alpha = \text{Int}, \beta = \text{Bool}$ :  $\rightarrow$  — тоже конструктор с двумя аргументами.

## Алгоритм унификации: правила (1)

- $C\ t_1 \dots t_k = C\ s_1 \dots s_k$  (одинаковые конструкторы): заменяется на  $t_1 = s_1, \dots, t_k = s_k$ .
  - $\text{Maybe } \alpha = \text{Maybe } [\text{Int}]$  даёт  $\alpha = [\text{Int}]$ .
  - $\alpha \rightarrow \beta = \text{Int} \rightarrow \text{Bool}$  даёт  $\alpha = \text{Int}, \beta = \text{Bool}$ :  $\rightarrow$  — тоже конструктор с двумя аргументами.
- $C \dots = D \dots$  (разные конструкторы):

## Алгоритм унификации: правила (1)

- $C\ t_1 \dots t_k = C\ s_1 \dots s_k$  (одинаковые конструкторы): заменяется на  $t_1 = s_1, \dots, t_k = s_k$ .
  - $\text{Maybe } \alpha = \text{Maybe } [\text{Int}]$  даёт  $\alpha = [\text{Int}]$ .
  - $\alpha \rightarrow \beta = \text{Int} \rightarrow \text{Bool}$  даёт  $\alpha = \text{Int}, \beta = \text{Bool}$ :  $\rightarrow$  — тоже конструктор с двумя аргументами.
- $C \dots = D \dots$  (разные конструкторы): унификатора не существует.

## Алгоритм унификации: правила (1)

- $C\ t_1 \dots t_k = C\ s_1 \dots s_k$  (одинаковые конструкторы): заменяется на  $t_1 = s_1, \dots, t_k = s_k$ .
  - $\text{Maybe } \alpha = \text{Maybe } [\text{Int}]$  даёт  $\alpha = [\text{Int}]$ .
  - $\alpha \rightarrow \beta = \text{Int} \rightarrow \text{Bool}$  даёт  $\alpha = \text{Int}, \beta = \text{Bool}$ :  $\rightarrow$  — тоже конструктор с двумя аргументами.
- $C \dots = D \dots$  (разные конструкторы): унификатора не существует.
  - $\text{Maybe } \alpha = [\beta], \text{Int} \rightarrow \text{Int} = \text{Int}$ .

## Алгоритм унификации: правила (2)

- $X = X$ :

## Алгоритм унификации: правила (2)

- $x = x$ : удаляется (можно обобщить и на  $t = t$ ).

## Алгоритм унификации: правила (2)

- $x = x$ : удаляется (можно обобщить и на  $t = t$ ).
- $x = t$  или  $t = x$ : два случая.

## Алгоритм унификации: правила (2)

- $x = x$ : удаляется (можно обобщить и на  $t = t$ ).
- $x = t$  или  $t = x$ : два случая.
  - Если в  $t$  нет переменной  $x$ :

## Алгоритм унификации: правила (2)

- $x = x$ : удаляется (можно обобщить и на  $t = t$ ).
- $x = t$  или  $t = x$ : два случая.
  - Если в  $t$  нет переменной  $x$ : во всех остальных уравнениях делаем замену  $x \mapsto t$ . Например,  $\alpha = [\beta]$ .
  - Если есть

## Алгоритм унификации: правила (2)

- $x = x$ : удаляется (можно обобщить и на  $t = t$ ).
- $x = t$  или  $t = x$ : два случая.
  - Если в  $t$  нет переменной  $x$ : во всех остальных уравнениях делаем замену  $x \mapsto t$ . Например,  $\alpha = [\beta]$ .
  - Если есть, то унификатора не существует:  $\alpha = [\alpha]$  требовало бы  $\alpha = [[[\dots]]]$ . В GHC это ошибка вида `Couldn't match expected type 't1' with actual type 't1 -> t2'`.

## Унификация: пример

$$\{ \alpha \rightarrow \beta = (\gamma \rightarrow \gamma) \rightarrow [\gamma], \quad \beta = [\text{Int}] \}$$

## Унификация: пример

$\{ \alpha \rightarrow \beta = (\gamma \rightarrow \gamma) \rightarrow [\gamma], \quad \beta = [\text{Int}] \}$

одинаковые конструкторы ( $\rightarrow$ )

## Унификация: пример

$$\{ \alpha \rightarrow \beta = (\gamma \rightarrow \gamma) \rightarrow [\gamma], \quad \beta = [\text{Int}] \}$$

одинаковые конструкторы ( $\rightarrow$ )

$$\{ \alpha = \gamma \rightarrow \gamma, \quad \beta = [\gamma], \quad \beta = [\text{Int}] \}$$

## Унификация: пример

$$\{ \alpha \rightarrow \beta = (\gamma \rightarrow \gamma) \rightarrow [\gamma], \quad \beta = [\text{Int}] \}$$

одинаковые конструкторы ( $\rightarrow$ )

$$\{ \alpha = \gamma \rightarrow \gamma, \quad \beta = [\gamma], \quad \beta = [\text{Int}] \}$$

$\alpha = t$ ,  $\alpha$  не входит в  $t$ :  $\sigma = \{ \alpha \mapsto \gamma \rightarrow \gamma \}$

## Унификация: пример

$$\{ \alpha \rightarrow \beta = (\gamma \rightarrow \gamma) \rightarrow [\gamma], \quad \beta = [\text{Int}] \}$$

одинаковые конструкторы ( $\rightarrow$ )

$$\{ \alpha = \gamma \rightarrow \gamma, \quad \beta = [\gamma], \quad \beta = [\text{Int}] \}$$

$\alpha = t$ ,  $\alpha$  не входит в  $t$ :  $\sigma = \{ \alpha \mapsto \gamma \rightarrow \gamma \}$

$$\{ \beta = [\gamma], \quad \beta = [\text{Int}] \}$$

## Унификация: пример

$$\{ \alpha \rightarrow \beta = (\gamma \rightarrow \gamma) \rightarrow [\gamma], \quad \beta = [\text{Int}] \}$$

одинаковые конструкторы ( $\rightarrow$ )

$$\{ \alpha = \gamma \rightarrow \gamma, \quad \beta = [\gamma], \quad \beta = [\text{Int}] \}$$

$\alpha = t$ ,  $\alpha$  не входит в  $t$ :  $\sigma = \{ \alpha \mapsto \gamma \rightarrow \gamma \}$

$$\{ \beta = [\gamma], \quad \beta = [\text{Int}] \}$$

$\beta = t$ : подстановка в остальные;  $\sigma = \{ \alpha \mapsto \gamma \rightarrow \gamma, \beta \mapsto [\gamma] \}$

## Унификация: пример

$\{ \alpha \rightarrow \beta = (\gamma \rightarrow \gamma) \rightarrow [\gamma], \quad \beta = [\text{Int}] \}$

одинаковые конструкторы ( $\rightarrow$ )

$\{ \alpha = \gamma \rightarrow \gamma, \quad \beta = [\gamma], \quad \beta = [\text{Int}] \}$

$\alpha = t$ ,  $\alpha$  не входит в  $t$ :  $\sigma = \{ \alpha \mapsto \gamma \rightarrow \gamma \}$

$\{ \beta = [\gamma], \quad \beta = [\text{Int}] \}$

$\beta = t$ : подстановка в остальные;  $\sigma = \{ \alpha \mapsto \gamma \rightarrow \gamma, \quad \beta \mapsto [\gamma] \}$

$\{ [\gamma] = [\text{Int}] \}$

## Унификация: пример

$\{ \alpha \rightarrow \beta = (\gamma \rightarrow \gamma) \rightarrow [\gamma], \quad \beta = [\text{Int}] \}$

одинаковые конструкторы ( $\rightarrow$ )

$\{ \alpha = \gamma \rightarrow \gamma, \quad \beta = [\gamma], \quad \beta = [\text{Int}] \}$

$\alpha = t$ ,  $\alpha$  не входит в  $t$ :  $\sigma = \{ \alpha \mapsto \gamma \rightarrow \gamma \}$

$\{ \beta = [\gamma], \quad \beta = [\text{Int}] \}$

$\beta = t$ : подстановка в остальные;  $\sigma = \{ \alpha \mapsto \gamma \rightarrow \gamma, \quad \beta \mapsto [\gamma] \}$

$\{ [\gamma] = [\text{Int}] \}$

одинаковые конструкторы ( $[]$ )

## Унификация: пример

$\{ \alpha \rightarrow \beta = (\gamma \rightarrow \gamma) \rightarrow [\gamma], \quad \beta = [\text{Int}] \}$

одинаковые конструкторы ( $\rightarrow$ )

$\{ \alpha = \gamma \rightarrow \gamma, \quad \beta = [\gamma], \quad \beta = [\text{Int}] \}$

$\alpha = t$ ,  $\alpha$  не входит в  $t$ :  $\sigma = \{ \alpha \mapsto \gamma \rightarrow \gamma \}$

$\{ \beta = [\gamma], \quad \beta = [\text{Int}] \}$

$\beta = t$ : подстановка в остальные;  $\sigma = \{ \alpha \mapsto \gamma \rightarrow \gamma, \quad \beta \mapsto [\gamma] \}$

$\{ [\gamma] = [\text{Int}] \}$

одинаковые конструкторы ( $[]$ )

$\{ \gamma = \text{Int} \}$

## Унификация: пример

$\{ \alpha \rightarrow \beta = (\gamma \rightarrow \gamma) \rightarrow [\gamma], \quad \beta = [\text{Int}] \}$

одинаковые конструкторы ( $\rightarrow$ )

$\{ \alpha = \gamma \rightarrow \gamma, \quad \beta = [\gamma], \quad \beta = [\text{Int}] \}$

$\alpha = t$ ,  $\alpha$  не входит в  $t$ :  $\sigma = \{ \alpha \mapsto \gamma \rightarrow \gamma \}$

$\{ \beta = [\gamma], \quad \beta = [\text{Int}] \}$

$\beta = t$ : подстановка в остальные;  $\sigma = \{ \alpha \mapsto \gamma \rightarrow \gamma, \beta \mapsto [\gamma] \}$

$\{ [\gamma] = [\text{Int}] \}$

одинаковые конструкторы ( $[]$ )

$\{ \gamma = \text{Int} \}$

$\gamma \mapsto \text{Int}$ , подставляем и в  $\sigma$

## Унификация: пример

$\{ \alpha \rightarrow \beta = (\gamma \rightarrow \gamma) \rightarrow [\gamma], \quad \beta = [\text{Int}] \}$

одинаковые конструкторы ( $\rightarrow$ )

$\{ \alpha = \gamma \rightarrow \gamma, \quad \beta = [\gamma], \quad \beta = [\text{Int}] \}$

$\alpha = t$ ,  $\alpha$  не входит в  $t$ :  $\sigma = \{ \alpha \mapsto \gamma \rightarrow \gamma \}$

$\{ \beta = [\gamma], \quad \beta = [\text{Int}] \}$

$\beta = t$ : подстановка в остальные;  $\sigma = \{ \alpha \mapsto \gamma \rightarrow \gamma, \beta \mapsto [\gamma] \}$

$\{ [\gamma] = [\text{Int}] \}$

одинаковые конструкторы ( $[]$ )

$\{ \gamma = \text{Int} \}$

$\gamma \mapsto \text{Int}$ , подставляем и в  $\sigma$

$\sigma = \{ \alpha \mapsto \text{Int} \rightarrow \text{Int}, \quad \beta \mapsto [\text{Int}], \quad \gamma \mapsto \text{Int} \}$

## Унификация: пример

$$\{ \alpha \rightarrow \beta = (\gamma \rightarrow \gamma) \rightarrow [\gamma], \quad \beta = [\text{Int}] \}$$

одинаковые конструкторы ( $\rightarrow$ )

$$\{ \alpha = \gamma \rightarrow \gamma, \quad \beta = [\gamma], \quad \beta = [\text{Int}] \}$$

$$\alpha = t, \alpha \text{ не входит в } t: \sigma = \{ \alpha \mapsto \gamma \rightarrow \gamma \}$$

$$\{ \beta = [\gamma], \quad \beta = [\text{Int}] \}$$

$$\beta = t: \text{подстановка в остальные}; \sigma = \{ \alpha \mapsto \gamma \rightarrow \gamma, \beta \mapsto [\gamma] \}$$

$$\{ [\gamma] = [\text{Int}] \}$$

одинаковые конструкторы ( $[]$ )

$$\{ \gamma = \text{Int} \}$$

$\gamma \mapsto \text{Int}$ , подставляем и в  $\sigma$

$$\sigma = \{ \alpha \mapsto \text{Int} \rightarrow \text{Int}, \quad \beta \mapsto [\text{Int}], \quad \gamma \mapsto \text{Int} \}$$

Проверка:

## Унификация: пример

$$\{ \alpha \rightarrow \beta = (\gamma \rightarrow \gamma) \rightarrow [\gamma], \quad \beta = [\text{Int}] \}$$

одинаковые конструкторы ( $\rightarrow$ )

$$\{ \alpha = \gamma \rightarrow \gamma, \quad \beta = [\gamma], \quad \beta = [\text{Int}] \}$$

$$\alpha = t, \alpha \text{ не входит в } t: \sigma = \{ \alpha \mapsto \gamma \rightarrow \gamma \}$$

$$\{ \beta = [\gamma], \quad \beta = [\text{Int}] \}$$

$$\beta = t: \text{подстановка в остальные}; \sigma = \{ \alpha \mapsto \gamma \rightarrow \gamma, \beta \mapsto [\gamma] \}$$

$$\{ [\gamma] = [\text{Int}] \}$$

одинаковые конструкторы ( $[]$ )

$$\{ \gamma = \text{Int} \}$$

$\gamma \mapsto \text{Int}$ , подставляем и в  $\sigma$

$$\sigma = \{ \alpha \mapsto \text{Int} \rightarrow \text{Int}, \quad \beta \mapsto [\text{Int}], \quad \gamma \mapsto \text{Int} \}$$

Проверка:  $(\text{Int} \rightarrow \text{Int}) \rightarrow [\text{Int}]$  и  $[\text{Int}]$  с обеих сторон.

## Унификация: пример без решения

$$\{ \alpha = [\beta], \quad \beta = \text{Maybe } \alpha \}$$

## Унификация: пример без решения

$$\{ \alpha = [\beta], \quad \beta = \text{Maybe } \alpha \}$$

$\alpha \mapsto [\beta]$ , подставляем во второе уравнение

## Унификация: пример без решения

$$\{ \alpha = [\beta], \quad \beta = \text{Maybe } \alpha \}$$

$\alpha \mapsto [\beta]$ , подставляем во второе уравнение

$$\{ \beta = \text{Maybe } [\beta] \}$$

## Унификация: пример без решения

$$\{ \alpha = [\beta], \quad \beta = \text{Maybe } \alpha \}$$

$\alpha \mapsto [\beta]$ , подставляем во второе уравнение

$$\{ \beta = \text{Maybe } [\beta] \}$$

$\beta$  входит в правую часть: решения нет.

# Унификация: пример без решения

$$\{ \alpha = [\beta], \quad \beta = \text{Maybe } \alpha \}$$

$\alpha \mapsto [\beta]$ , подставляем во второе уравнение

$$\{ \beta = \text{Maybe } [\beta] \}$$

$\beta$  входит в правую часть: решения нет.

- Occurs check срабатывает не сразу, а после подстановки.
- Это в миниатюре **f3**  $x = x \ x$  со слайда с примерами.

## Алгоритм $\mathcal{J}$ вывода типов

- $\Gamma$  — окружение типов (набор переменных с их типами).
- $\mathcal{J}(\Gamma; e)$  возвращает наиболее общий тип для  $e$  с окружением  $\Gamma$  или выдаёт ошибку.

## Алгоритм $\mathcal{T}$ вывода типов

- $\Gamma$  — окружение типов (набор переменных с их типами).
- $\mathcal{T}(\Gamma; e)$  возвращает наиболее общий тип для  $e$  с окружением  $\Gamma$  или выдаёт ошибку.
- $\phi$  — глобальная переменная (только для простоты объяснения), содержащая уже сделанные подстановки.
- Функция *fresh*() возвращает свежую переменную типа (т.е. такую, которой ещё нигде не было).
- Функция *ftv* возвращает свободные переменные типов в аргументе.
- Функция *unify* решает задачу унификации.

## Алгоритм $\mathcal{J}$ : приведение определений и выражений к стандартному виду

- Переименуем связанные переменные с одинаковыми именами.
- Заменяем `where` на `let`.
- Заменяем `if` и сопоставления с образцом в уравнениях на `case`.

# Приведение к стандартному виду: пример

- Пример:

```
not' True  = False
```

```
not' False = True
```

```
g x = h x where h x = not' x
```

# Приведение к стандартному виду: пример

- Пример:

```
not' True  = False
```

```
not' False = True
```

```
g x = h x where h x = not' x
```

превращается в

```
not' = \x -> case x of
```

```
    True  -> False
```

```
    False -> True
```

```
g = \x -> let h = \y -> not' y in h x
```

## Алгоритм $\mathcal{J}$ : переменная

- Переменная  $x$ : тип берётся из окружения (если  $x$  там нет, это ошибка типизации).
- Если он полиморфный, то переменные под кванторами заменяются на свежие (новые).

## Алгоритм $\mathcal{I}$ : переменная

- Переменная  $x$ : тип берётся из окружения (если  $x$  там нет, это ошибка типизации).
- Если он полиморфный, то переменные под кванторами заменяются на свежие (новые).
- Примеры (в  $\Gamma$ :  $\text{not} : \text{Bool} \rightarrow \text{Bool}$ ,  $\text{id} : \forall \alpha. \alpha \rightarrow \alpha$ ; числовые литералы до обсуждения классов типов считаем  $\text{Int}$ ):
  - $\text{not} : \text{Bool} \rightarrow \text{Bool}$  как есть.

## Алгоритм $\mathcal{I}$ : переменная

- Переменная  $x$ : тип берётся из окружения (если  $x$  там нет, это ошибка типизации).
- Если он полиморфный, то переменные под кванторами заменяются на свежие (новые).
- Примеры (в  $\Gamma$ :  $\text{not} : \text{Bool} \rightarrow \text{Bool}$ ,  $\text{id} : \forall \alpha. \alpha \rightarrow \alpha$ ; числовые литералы до обсуждения классов типов считаем  $\text{Int}$ ):
  - $\text{not}$ :  $\text{Bool} \rightarrow \text{Bool}$  как есть.
  - $\text{id}$ :  $\alpha_1 \rightarrow \alpha_1$ ; второе вхождение  $\text{id}$  в том же выражении —  $\alpha_2 \rightarrow \alpha_2$ . Каждое использование типизируется независимо.

## Алгоритм $\mathcal{J}$ : применение

- Применение функции  $e_1$   $e_2$ :
  - Выводим тип  $t_1$  для  $e_1$  и тип  $t_2$  для  $e_2$ .
  - $t_1$  должен иметь вид  $t_2 \rightarrow \alpha$  (где  $\alpha$  — свежая переменная). Унифицируем их.
  - Результат (тип  $e_1$   $e_2$ ):

## Алгоритм $\mathcal{J}$ : применение

- Применение функции  $e_1 e_2$ :
  - Выводим тип  $t_1$  для  $e_1$  и тип  $t_2$  для  $e_2$ .
  - $t_1$  должен иметь вид  $t_2 \rightarrow \alpha$  (где  $\alpha$  — свежая переменная). Унифицируем их.
  - Результат (тип  $e_1 e_2$ ):  $\alpha$  (после подстановок).

## Алгоритм $\mathcal{J}$ : применение

- Применение функции  $e_1 e_2$ :
  - Выводим тип  $t_1$  для  $e_1$  и тип  $t_2$  для  $e_2$ .
  - $t_1$  должен иметь вид  $t_2 \rightarrow \alpha$  (где  $\alpha$  — свежая переменная). Унифицируем их.
  - Результат (тип  $e_1 e_2$ ):  $\alpha$  (после подстановок).
- Примеры:
  - `not True`:  $t_1 = \text{Bool} \rightarrow \text{Bool}$ ,  $t_2 = \text{Bool}$ ;  
унифицируем  $\text{Bool} \rightarrow \text{Bool} = \text{Bool} \rightarrow \alpha$ :  $\alpha = \text{Bool}$ .

## Алгоритм $\mathcal{J}$ : применение

- Применение функции  $e_1 e_2$ :
  - Выводим тип  $t_1$  для  $e_1$  и тип  $t_2$  для  $e_2$ .
  - $t_1$  должен иметь вид  $t_2 \rightarrow \alpha$  (где  $\alpha$  — свежая переменная). Унифицируем их.
  - Результат (тип  $e_1 e_2$ ):  $\alpha$  (после подстановок).
- Примеры:
  - **not True**:  $t_1 = \text{Bool} \rightarrow \text{Bool}$ ,  $t_2 = \text{Bool}$ ;  
унифицируем  $\text{Bool} \rightarrow \text{Bool} = \text{Bool} \rightarrow \alpha$ :  $\alpha = \text{Bool}$ .
  - **id id**:  $t_1 = \alpha_1 \rightarrow \alpha_1$ ,  $t_2 = \alpha_2 \rightarrow \alpha_2$ ; унифицируем  
 $\alpha_1 \rightarrow \alpha_1 = (\alpha_2 \rightarrow \alpha_2) \rightarrow \alpha$ :  $\alpha_1 = \alpha_2 \rightarrow \alpha_2$ ,  $\alpha = \alpha_2 \rightarrow \alpha_2$ .  
Результат  $\alpha_2 \rightarrow \alpha_2$ .

## Алгоритм $\mathcal{I}$ : применение

- Применение функции  $e_1 e_2$ :
  - Выводим тип  $t_1$  для  $e_1$  и тип  $t_2$  для  $e_2$ .
  - $t_1$  должен иметь вид  $t_2 \rightarrow \alpha$  (где  $\alpha$  — свежая переменная). Унифицируем их.
  - Результат (тип  $e_1 e_2$ ):  $\alpha$  (после подстановок).
- Примеры:
  - **not True**:  $t_1 = \text{Bool} \rightarrow \text{Bool}$ ,  $t_2 = \text{Bool}$ ;  
унифицируем  $\text{Bool} \rightarrow \text{Bool} = \text{Bool} \rightarrow \alpha$ :  $\alpha = \text{Bool}$ .
  - **id id**:  $t_1 = \alpha_1 \rightarrow \alpha_1$ ,  $t_2 = \alpha_2 \rightarrow \alpha_2$ ; унифицируем  
 $\alpha_1 \rightarrow \alpha_1 = (\alpha_2 \rightarrow \alpha_2) \rightarrow \alpha$ :  $\alpha_1 = \alpha_2 \rightarrow \alpha_2$ ,  $\alpha = \alpha_2 \rightarrow \alpha_2$ .  
Результат  $\alpha_2 \rightarrow \alpha_2$ .
  - **not id**: унифицируем  
 $\text{Bool} \rightarrow \text{Bool} = (\alpha_2 \rightarrow \alpha_2) \rightarrow \alpha$ ; нужно  
 $\text{Bool} = \alpha_2 \rightarrow \alpha_2$ , а это разные конструкторы:  
ошибка типа.

## Алгоритм $\mathcal{J}$ : лямбда-выражение

- Лямбда-выражение  $\lambda x.e_1$ :
  - Тип  $x$  — свежая переменная  $\alpha$ .  $x : \alpha$  добавляется в окружение (на время этого вывода).
  - Выводится тип  $t_1$  для  $e_1$ .
  - Результат:

## Алгоритм $\mathcal{J}$ : лямбда-выражение

- Лямбда-выражение  $\lambda x.e_1$ :
  - Тип  $x$  — свежая переменная  $\alpha$ .  $x : \alpha$  добавляется в окружение (на время этого вывода).
  - Выводится тип  $t_1$  для  $e_1$ .
  - Результат:  $\alpha \rightarrow t_1$ .

## Алгоритм $\mathcal{J}$ : лямбда-выражение

- Лямбда-выражение  $\lambda x.e_1$ :
  - Тип  $x$  — свежая переменная  $\alpha$ .  $x : \alpha$  добавляется в окружение (на время этого вывода).
  - Выводится тип  $t_1$  для  $e_1$ .
  - Результат:  $\alpha \rightarrow t_1$ .
- Примеры:
  - $\backslash x \rightarrow \text{not } x : x : \alpha$ ; для  $\text{not}$   $x$  унифицируем  $\text{Bool} \rightarrow \text{Bool} = \alpha \rightarrow \beta : \alpha = \text{Bool}, \beta = \text{Bool}$ .  
Результат  $\alpha \rightarrow \beta = \text{Bool} \rightarrow \text{Bool}$ .

## Алгоритм $\mathcal{J}$ : лямбда-выражение

- Лямбда-выражение  $\lambda x.e_1$ :
  - Тип  $x$  — свежая переменная  $\alpha$ .  $x : \alpha$  добавляется в окружение (на время этого вывода).
  - Выводится тип  $t_1$  для  $e_1$ .
  - Результат:  $\alpha \rightarrow t_1$ .
- Примеры:
  - $\backslash x \rightarrow \text{not } x : x : \alpha$ ; для  $\text{not}$   $x$  унифицируем  $\text{Bool} \rightarrow \text{Bool} = \alpha \rightarrow \beta : \alpha = \text{Bool}, \beta = \text{Bool}$ .  
Результат  $\alpha \rightarrow \beta = \text{Bool} \rightarrow \text{Bool}$ .
  - $\backslash x \rightarrow x : x : \alpha$ , результат  $\alpha \rightarrow \alpha$ . Без  $\forall$ : обобщение делает только  $\text{let}$ .

## Алгоритм $\mathcal{J}$ : **let**

- **let**-выражение **let**  $x = e_1$  **in**  $e_2$ :
  - Выводим тип  $t_1$  для  $e_1$ .
  - Тип  $x$  получается навешиванием  $\forall$  на свободные переменные  $\phi(t_1)$ , не входящие свободно в  $\phi(\Gamma)$ . Добавляем его в окружение (тоже на время этого вывода).
  - Выводим тип  $t_2$  для  $e_2$ . Результат:

## Алгоритм $\mathcal{J}$ : **let**

- **let**-выражение **let**  $x = e_1$  **in**  $e_2$ :
  - Выводим тип  $t_1$  для  $e_1$ .
  - Тип  $x$  получается навешиванием  $\forall$  на свободные переменные  $\phi(t_1)$ , не входящие свободно в  $\phi(\Gamma)$ . Добавляем его в окружение (тоже на время этого вывода).
  - Выводим тип  $t_2$  для  $e_2$ . Результат:  $t_2$ .

## Алгоритм $\mathcal{J}$ : **let**

- **let**-выражение **let**  $x = e_1$  **in**  $e_2$ :
  - Выводим тип  $t_1$  для  $e_1$ .
  - Тип  $x$  получается навешиванием  $\forall$  на свободные переменные  $\phi(t_1)$ , не входящие свободно в  $\phi(\Gamma)$ . Добавляем его в окружение (тоже на время этого вывода).
  - Выводим тип  $t_2$  для  $e_2$ . Результат:  $t_2$ .
- Пример: **let**  $f = \lambda x \rightarrow x$  **in** ( $f$  **1**,  $f$  **True**).
  - $t_1 = \alpha \rightarrow \alpha$ ,  $\alpha$  нет в  $\Gamma$ , поэтому  $f : \forall \alpha. \alpha \rightarrow \alpha$ .

## Алгоритм $\mathcal{J}$ : **let**

- **let**-выражение **let**  $x = e_1$  **in**  $e_2$ :
  - Выводим тип  $t_1$  для  $e_1$ .
  - Тип  $x$  получается навешиванием  $\forall$  на свободные переменные  $\phi(t_1)$ , не входящие свободно в  $\phi(\Gamma)$ . Добавляем его в окружение (тоже на время этого вывода).
  - Выводим тип  $t_2$  для  $e_2$ . Результат:  $t_2$ .
- Пример: **let**  $f = \lambda x \rightarrow x$  **in** ( $f$  **1**,  $f$  **True**).
  - $t_1 = \alpha \rightarrow \alpha$ ,  $\alpha$  нет в  $\Gamma$ , поэтому  $f : \forall \alpha. \alpha \rightarrow \alpha$ .
  - В теле  $f$  **1** даёт  $\text{Int}$ ,  $f$  **True** —  $\text{Bool}$ ; результат  $(\text{Int}, \text{Bool})$ .

## Алгоритм $\mathcal{J}$ : **let**

- **let**-выражение **let**  $x = e_1$  **in**  $e_2$ :
  - Выводим тип  $t_1$  для  $e_1$ .
  - Тип  $x$  получается навешиванием  $\forall$  на свободные переменные  $\phi(t_1)$ , не входящие свободно в  $\phi(\Gamma)$ . Добавляем его в окружение (тоже на время этого вывода).
  - Выводим тип  $t_2$  для  $e_2$ . Результат:  $t_2$ .
- Пример: **let**  $f = \lambda x \rightarrow x$  **in** ( $f$  1,  $f$  True).
  - $t_1 = \alpha \rightarrow \alpha$ ,  $\alpha$  нет в  $\Gamma$ , поэтому  $f : \forall \alpha. \alpha \rightarrow \alpha$ .
  - В теле  $f$  1 даёт Int,  $f$  True — Bool; результат (Int, Bool).
  - А  $(\lambda f \rightarrow (f$  1,  $f$  True))  $(\lambda x \rightarrow x)$  не типизируется:  $f : \beta$  мономорфна, и  $\beta$  пришлось бы унифицировать и с  $\text{Int} \rightarrow \gamma_1$ , и с  $\text{Bool} \rightarrow \gamma_2$ .

## Алгоритм $\mathcal{J}$ : рекурсивный **let**

- Особый случай: рекурсивный **let** ( $x$  входит в  $e_1$ ).

## Алгоритм $\mathcal{I}$ : рекурсивный **let**

- Особый случай: рекурсивный **let** ( $x$  входит в  $e_1$ ).
  - **fix**  $f$  возвращает неподвижную точку  $f$  (т.е.  $f$  (**fix**  $f$ ) = **fix**  $f$ ).

## Алгоритм $\mathcal{I}$ : рекурсивный **let**

- Особый случай: рекурсивный **let** ( $x$  входит в  $e_1$ ).
  - **fix**  $f$  возвращает неподвижную точку  $f$  (т.е.  $f$  ( $\text{fix } f$ ) = **fix**  $f$ ).
  - Соответственно, её тип

## Алгоритм $\mathcal{I}$ : рекурсивный **let**

- Особый случай: рекурсивный **let** ( $x$  входит в  $e_1$ ).
  - **fix**  $f$  возвращает неподвижную точку  $f$  (т.е.  $f$  ( $\text{fix } f$ ) = **fix**  $f$ ).
  - Соответственно, её тип **fix**  $:: \text{forall } a. (a \rightarrow a) \rightarrow a$ .

## Алгоритм $\mathcal{J}$ : рекурсивный **let**

- Особый случай: рекурсивный **let** ( $x$  входит в  $e_1$ ).
  - **fix**  $f$  возвращает неподвижную точку  $f$  (т.е.  $f$  ( $\text{fix } f$ ) = **fix**  $f$ ).
  - Соответственно, её тип **fix**  $:: \text{forall } a. (a \rightarrow a) \rightarrow a$ .
  - **let**  $x = e_1$  **in**  $e_2$  превращается в **let**  $x = \text{fix } (\lambda x \rightarrow e_1)$  **in**  $e_2$ .
  - Этот **let** не рекурсивный:

## Алгоритм $\mathcal{I}$ : рекурсивный **let**

- Особый случай: рекурсивный **let** ( $x$  входит в  $e_1$ ).
  - **fix**  $f$  возвращает неподвижную точку  $f$  (т.е.  $f$  ( $\text{fix } f$ ) = **fix**  $f$ ).
  - Соответственно, её тип **fix**  $:: \text{forall } a. (a \rightarrow a) \rightarrow a$ .
  - **let**  $x = e_1$  **in**  $e_2$  превращается в **let**  $x = \text{fix } (\lambda x \rightarrow e_1)$  **in**  $e_2$ .
  - Этот **let** не рекурсивный:  $x$  в **let**  $x = \dots$  и в  $\lambda x \rightarrow e_1$  — разные!

## Алгоритм $\mathcal{I}$ : рекурсивный **let**

- Особый случай: рекурсивный **let** ( $x$  входит в  $e_1$ ).
  - **fix**  $f$  возвращает неподвижную точку  $f$  (т.е.  $f$  ( $\text{fix } f$ ) = **fix**  $f$ ).
  - Соответственно, её тип **fix**  $:: \text{forall } a. (a \rightarrow a) \rightarrow a$ .
  - **let**  $x = e_1$  **in**  $e_2$  превращается в **let**  $x = \text{fix } (\lambda x \rightarrow e_1)$  **in**  $e_2$ .
  - Этот **let** не рекурсивный:  $x$  в **let**  $x = \dots$  и в  $\lambda x \rightarrow e_1$  — разные!
- Пример: **let**  $\text{ones} = 1 : \text{ones}$  **in**  $\dots \rightsquigarrow \text{fix } (\lambda \text{ones} \rightarrow 1 : \text{ones})$ .
  - Внутри лямбды  $\text{ones} : \alpha$ ;  $c (:) : \forall \beta. \beta \rightarrow [\beta] \rightarrow [\beta]$  в  $\Gamma$  тип  $1 : \text{ones} \text{ — } [\text{Int}]$ , причём  $\alpha = [\text{Int}]$ .

## Алгоритм $\mathcal{I}$ : рекурсивный **let**

- Особый случай: рекурсивный **let** ( $x$  входит в  $e_1$ ).
  - fix**  $f$  возвращает неподвижную точку  $f$  (т.е.  $f$  ( $\text{fix } f$ ) = **fix**  $f$ ).
  - Соответственно, её тип **fix**  $:: \text{forall } a. (a \rightarrow a) \rightarrow a$ .
  - let**  $x = e_1$  **in**  $e_2$  превращается в **let**  $x = \text{fix } (\lambda x \rightarrow e_1)$  **in**  $e_2$ .
  - Этот **let** не рекурсивный:  $x$  в **let**  $x = \dots$  и в  $\lambda x \rightarrow e_1$  — разные!
- Пример: **let**  $\text{ones} = 1 : \text{ones}$  **in**  $\dots \rightsquigarrow \text{fix } (\lambda \text{ones} \rightarrow 1 : \text{ones})$ .
  - Внутри лямбды  $\text{ones} : \alpha$ ; с  $(:) : \forall \beta. \beta \rightarrow [\beta] \rightarrow [\beta]$  в  $\Gamma$  тип  $1 : \text{ones} \text{ — } [\text{Int}]$ , причём  $\alpha = [\text{Int}]$ .
  - Лямбда:  $[\text{Int}] \rightarrow [\text{Int}]$ ; **fix** даёт  $[\text{Int}]$ .

## Алгоритм $\mathcal{J}$ : **case**

- **case**  $e$  **of**  $pat_1 \rightarrow body_1; pat_2 \rightarrow body_2; \dots$

## Алгоритм $\mathcal{I}$ : **case**

- **case**  $e$  **of**  $pat_1 \rightarrow body_1; pat_2 \rightarrow body_2; \dots$ 
  - Выводим тип  $t$  для  $e$ .
  - Для каждой ветви:
    - свежие переменные типов для переменных в  $pat_i$ ;
    - тип  $pat_i$  унифицируем с  $t$ ;
    - выводим тип  $t_i$  для  $body_i$ .
  - Унифицируем все  $t_i$ . Результат:

## Алгоритм $\mathcal{J}$ : **case**

- **case**  $e$  **of**  $pat_1 \rightarrow body_1; pat_2 \rightarrow body_2; \dots$ 
  - Выводим тип  $t$  для  $e$ .
  - Для каждой ветви:
    - свежие переменные типов для переменных в  $pat_i$ ;
    - тип  $pat_i$  унифицируем с  $t$ ;
    - выводим тип  $t_i$  для  $body_i$ .
  - Унифицируем все  $t_i$ . Результат:  $t_1$ .

## Алгоритм $\mathcal{I}$ : **case**

- **case**  $e$  **of**  $pat_1 \rightarrow body_1; pat_2 \rightarrow body_2; \dots$ 
  - Выводим тип  $t$  для  $e$ .
  - Для каждой ветви:
    - свежие переменные типов для переменных в  $pat_i$ ;
    - тип  $pat_i$  унифицируем с  $t$ ;
    - выводим тип  $t_i$  для  $body_i$ .
  - Унифицируем все  $t_i$ . Результат:  $t_1$ .
- Пример:  $\backslash m \rightarrow$  **case**  $m$  **of**  
    **Nothing**  $\rightarrow 1$   
    **Just**  $x \rightarrow x$ 
  - $m : \alpha$ . Ветвь 1: **Nothing** имеет тип  $\text{Maybe } \beta_1$ , унифицируем с  $\alpha$ ; тело —  $\text{Int}$ .

## Алгоритм $\mathcal{I}$ : **case**

- **case**  $e$  **of**  $pat_1 \rightarrow body_1; pat_2 \rightarrow body_2; \dots$ 
  - Выводим тип  $t$  для  $e$ .
  - Для каждой ветви:
    - свежие переменные типов для переменных в  $pat_i$ ;
    - тип  $pat_i$  унифицируем с  $t$ ;
    - выводим тип  $t_i$  для  $body_i$ .
  - Унифицируем все  $t_i$ . Результат:  $t_1$ .
- Пример:  $\backslash m \rightarrow$  **case**  $m$  **of**  
    **Nothing**  $\rightarrow 1$   
    **Just**  $x \rightarrow x$ 
  - $m : \alpha$ . Ветвь 1: **Nothing** имеет тип **Maybe**  $\beta_1$ , унифицируем с  $\alpha$ ; тело — **Int**.
  - Ветвь 2:  $x : \beta_2$ , образец **Just**  $x$  — **Maybe**  $\beta_2$ , унифицируем с **Maybe**  $\beta_1$ :  $\beta_1 = \beta_2$ ; тело —  $\beta_2$ .

## Алгоритм $\mathcal{I}$ : **case**

- **case**  $e$  **of**  $pat_1 \rightarrow body_1; pat_2 \rightarrow body_2; \dots$ 
  - Выводим тип  $t$  для  $e$ .
  - Для каждой ветви:
    - свежие переменные типов для переменных в  $pat_i$ ;
    - тип  $pat_i$  унифицируем с  $t$ ;
    - выводим тип  $t_i$  для  $body_i$ .
  - Унифицируем все  $t_i$ . Результат:  $t_1$ .
- Пример:  $\backslash m \rightarrow$  **case**  $m$  **of**
  - Nothing**  $\rightarrow 1$
  - Just**  $x \rightarrow x$
  - $m : \alpha$ . Ветвь 1: **Nothing** имеет тип  $\text{Maybe } \beta_1$ , унифицируем с  $\alpha$ ; тело —  $\text{Int}$ .
  - Ветвь 2:  $x : \beta_2$ , образец **Just**  $x$  —  $\text{Maybe } \beta_2$ , унифицируем с  $\text{Maybe } \beta_1$ :  $\beta_1 = \beta_2$ ; тело —  $\beta_2$ .
  - Унифицируем тела:  $\beta_2 = \text{Int}$ , значит  $\alpha = \text{Maybe Int}$ ; итог —  $\text{Maybe Int} \rightarrow \text{Int}$ .

## Примеры

- Определения уровня модуля обрабатываем как  $e_1$  в `let`  $e_1$  `in`  $e_2$ : выводим тип, обобщаем, добавляем в  $\Gamma$  для следующих определений.

## Примеры

- Определения уровня модуля обрабатываем как  $e_1$  в `let`  $e_1$  `in`  $e_2$ : выводим тип, обобщаем, добавляем в  $\Gamma$  для следующих определений.
- Разберём несколько примеров

# Примеры

- Определения уровня модуля обрабатываем как  $e_1$  в `let  $e_1$  in  $e_2$` : выводим тип, обобщаем, добавляем в  $\Gamma$  для следующих определений.
- Разберём несколько примеров
- Без `let`:  
`f1 = (.) (.)`

# Примеры

- Определения уровня модуля обрабатываем как  $e_1$  в `let  $e_1$  in  $e_2$` : выводим тип, обобщаем, добавляем в  $\Gamma$  для следующих определений.
- Разберём несколько примеров
- Без `let`:

```
f1 = (.) (.)
```

```
f1 :: (a1 -> b -> c) -> a1 -> (a2 -> b) -> a2 -> c
```

# Примеры

- Определения уровня модуля обрабатываем как  $e_1$  в `let  $e_1$  in  $e_2$` : выводим тип, обобщаем, добавляем в  $\Gamma$  для следующих определений.
- Разберём несколько примеров
- Без `let`:  
`f1 = (.) (.)`  
`f1 :: (a1 -> b -> c) -> a1 -> (a2 -> b) -> a2 -> c`
- С `let`-полиморфизмом:  
`f2 x = let pair x = (x,x) in pair (pair x)`

# Примеры

- Определения уровня модуля обрабатываем как  $e_1$  в `let  $e_1$  in  $e_2$` : выводим тип, обобщаем, добавляем в  $\Gamma$  для следующих определений.
- Разберём несколько примеров

- Без `let`:

```
f1 = (.) (.)
```

```
f1 :: (a1 -> b -> c) -> a1 -> (a2 -> b) -> a2 -> c
```

- С `let`-полиморфизмом:

```
f2 x = let pair x = (x,x) in pair (pair x)
```

```
f2 :: b -> ((b, b), (b, b))
```

# Примеры

- Определения уровня модуля обрабатываем как  $e_1$  в `let  $e_1$  in  $e_2$` : выводим тип, обобщаем, добавляем в  $\Gamma$  для следующих определений.

- Разберём несколько примеров

- Без `let`:

```
f1 = (.) (.)
```

```
f1 :: (a1 -> b -> c) -> a1 -> (a2 -> b) -> a2 -> c
```

- С `let`-полиморфизмом:

```
f2 x = let pair x = (x,x) in pair (pair x)
```

```
f2 :: b -> ((b, b), (b, b))
```

- Пример ошибки типа:

```
f3 x = x x
```

# Примеры

- Определения уровня модуля обрабатываем как  $e_1$  в `let  $e_1$  in  $e_2$` : выводим тип, обобщаем, добавляем в  $\Gamma$  для следующих определений.

- Разберём несколько примеров

- Без `let`:

```
f1 = (.) (.)
```

```
f1 :: (a1 -> b -> c) -> a1 -> (a2 -> b) -> a2 -> c
```

- С `let`-полиморфизмом:

```
f2 x = let pair x = (x,x) in pair (pair x)
```

```
f2 :: b -> ((b, b), (b, b))
```

- Пример ошибки типа:

```
f3 x = x x
```

```
Couldn't match expected type 't1'
```

```
with actual type 't1 -> t2'
```

(тип `x` был бы бесконечным).

## А как же классы типов?

- Мы выводили типы без ограничений, но  $1 + x$  требует `Num`.

## А как же классы типов?

- Мы выводили типы без ограничений, но `1 + x` требует `Num`.
- Расширение системы Хиндли — Милнера — *квалифицированные типы*: кроме подстановки  $\phi$  накапливается набор ограничений; при обобщении в `let` они попадают в контекст: `Num`  
`a => a -> a`.

## А как же классы типов?

- Мы выводили типы без ограничений, но `1 + x` требует `Num`.
- Расширение системы Хиндли — Милнера — *квалифицированные типы*: кроме подстановки  $\phi$  накапливается набор ограничений; при обобщении в `let` они попадают в контекст: `Num a => a -> a`.
- Реализация — передача «словарей»: `Num a => a -> a` компилируется примерно в `NumDict a -> a -> a`.

## Классы типов: пример

- Выведем тип  $\lambda x. x + 1$ . В  $\Gamma$ :  
 $(+) : \forall \alpha. \text{Num } \alpha \Rightarrow \alpha \rightarrow \alpha \rightarrow \alpha, 1 : \forall \alpha. \text{Num } \alpha \Rightarrow \alpha.$

## Классы типов: пример

- Выведем тип  $\lambda x. x + 1$ . В  $\Gamma$ :  
 $(+) : \forall \alpha. \text{Num } \alpha \Rightarrow \alpha \rightarrow \alpha \rightarrow \alpha, 1 : \forall \alpha. \text{Num } \alpha \Rightarrow \alpha$ .
  - $(+) : \alpha_1 \rightarrow \alpha_1 \rightarrow \alpha_1$ , в набор ограничений  $S$  добавляется  $\text{Num } \alpha_1$ .

## Классы типов: пример

- Выведем тип  $\backslash x \rightarrow x + 1$ . В  $\Gamma$ :  
 $(+) : \forall \alpha. \text{Num } \alpha \Rightarrow \alpha \rightarrow \alpha \rightarrow \alpha, 1 : \forall \alpha. \text{Num } \alpha \Rightarrow \alpha$ .
  - $(+): \alpha_1 \rightarrow \alpha_1 \rightarrow \alpha_1$ , в набор ограничений  $S$  добавляется  $\text{Num } \alpha_1$ .
  - $x : \beta; \backslash x + 1$  это  $(+) \ x \ 1$ . Унифицируем  $\alpha_1 \rightarrow \alpha_1 \rightarrow \alpha_1 = \beta \rightarrow \gamma: \beta = \alpha_1, \gamma = \alpha_1 \rightarrow \alpha_1$ .

## Классы типов: пример

- Выведем тип  $\backslash x \rightarrow x + 1$ . В Г:  
 $(+) : \forall \alpha. \text{Num } \alpha \Rightarrow \alpha \rightarrow \alpha \rightarrow \alpha, 1 : \forall \alpha. \text{Num } \alpha \Rightarrow \alpha$ .
  - $(+) : \alpha_1 \rightarrow \alpha_1 \rightarrow \alpha_1$ , в набор ограничений  $C$  добавляется  $\text{Num } \alpha_1$ .
  - $x : \beta; x + 1$  это  $(+) \ x \ 1$ . Унифицируем  $\alpha_1 \rightarrow \alpha_1 \rightarrow \alpha_1 = \beta \rightarrow \gamma : \beta = \alpha_1, \gamma = \alpha_1 \rightarrow \alpha_1$ .
  - $1 : \alpha_2$ , в  $C$  добавляется  $\text{Num } \alpha_2$ . Унифицируем  $\alpha_1 \rightarrow \alpha_1 = \alpha_2 \rightarrow \delta : \alpha_2 = \alpha_1, \delta = \alpha_1$ .

## Классы типов: пример

- Выведем тип  $\lambda x. x + 1$ . В  $\Gamma$ :  
 $(+) : \forall \alpha. \text{Num } \alpha \Rightarrow \alpha \rightarrow \alpha \rightarrow \alpha, 1 : \forall \alpha. \text{Num } \alpha \Rightarrow \alpha$ .
  - $(+)$ :  $\alpha_1 \rightarrow \alpha_1 \rightarrow \alpha_1$ , в набор ограничений  $C$  добавляется  $\text{Num } \alpha_1$ .
  - $x : \beta; x + 1$  это  $(+) \ x \ 1$ . Унифицируем  $\alpha_1 \rightarrow \alpha_1 \rightarrow \alpha_1 = \beta \rightarrow \gamma: \beta = \alpha_1, \gamma = \alpha_1 \rightarrow \alpha_1$ .
  - $1 : \alpha_2$ , в  $C$  добавляется  $\text{Num } \alpha_2$ . Унифицируем  $\alpha_1 \rightarrow \alpha_1 = \alpha_2 \rightarrow \delta: \alpha_2 = \alpha_1, \delta = \alpha_1$ .
  - Лямбда:  $\alpha_1 \rightarrow \alpha_1$ ; после подстановки  $C = \{\text{Num } \alpha_1\}$ .

## Классы типов: пример

- Выведем тип  $\lambda x. x + 1$ . В  $\Gamma$ :  
 $(+) : \forall \alpha. \text{Num } \alpha \Rightarrow \alpha \rightarrow \alpha \rightarrow \alpha, 1 : \forall \alpha. \text{Num } \alpha \Rightarrow \alpha$ .
  - $(+): \alpha_1 \rightarrow \alpha_1 \rightarrow \alpha_1$ , в набор ограничений  $C$  добавляется  $\text{Num } \alpha_1$ .
  - $x : \beta; x + 1$  это  $(+) x 1$ . Унифицируем  $\alpha_1 \rightarrow \alpha_1 \rightarrow \alpha_1 = \beta \rightarrow \gamma: \beta = \alpha_1, \gamma = \alpha_1 \rightarrow \alpha_1$ .
  - $1: \alpha_2$ , в  $C$  добавляется  $\text{Num } \alpha_2$ . Унифицируем  $\alpha_1 \rightarrow \alpha_1 = \alpha_2 \rightarrow \delta: \alpha_2 = \alpha_1, \delta = \alpha_1$ .
  - Лямбда:  $\alpha_1 \rightarrow \alpha_1$ ; после подстановки  $C = \{\text{Num } \alpha_1\}$ .
  - Обобщение: ограничения на  $\alpha_1$  уходят в контекст —  $\text{Num } a \Rightarrow a \rightarrow a$ .

## Классы типов: пример

- Выведем тип  $\backslash x \rightarrow x + 1$ . В  $\Gamma$ :  
 $(+) : \forall \alpha. \text{Num } \alpha \Rightarrow \alpha \rightarrow \alpha \rightarrow \alpha, 1 : \forall \alpha. \text{Num } \alpha \Rightarrow \alpha$ .
  - $(+)$ :  $\alpha_1 \rightarrow \alpha_1 \rightarrow \alpha_1$ , в набор ограничений  $C$  добавляется  $\text{Num } \alpha_1$ .
  - $x : \beta; x + 1$  это  $(+) x 1$ . Унифицируем  $\alpha_1 \rightarrow \alpha_1 \rightarrow \alpha_1 = \beta \rightarrow \gamma : \beta = \alpha_1, \gamma = \alpha_1 \rightarrow \alpha_1$ .
  - $1 : \alpha_2$ , в  $C$  добавляется  $\text{Num } \alpha_2$ . Унифицируем  $\alpha_1 \rightarrow \alpha_1 = \alpha_2 \rightarrow \delta : \alpha_2 = \alpha_1, \delta = \alpha_1$ .
  - Лямбда:  $\alpha_1 \rightarrow \alpha_1$ ; после подстановки  $C = \{\text{Num } \alpha_1\}$ .
  - Обобщение: ограничения на  $\alpha_1$  уходят в контекст —  $\text{Num } a \Rightarrow a \rightarrow a$ .
- А для  $\text{True} + 1$  получаем  $\alpha_1 = \text{Bool}$  и  $\text{Num Bool}$  без экземпляра — ошибка `No instance for 'Num Bool'` из лекции 1.

## Неоднозначные типы и умолчания

- Проблема: если переменная типа осталась *только* в ограничениях, тип неоднозначен. Какой тип у `read s` в `show (read s)`?

## Неоднозначные типы и умолчания

- Проблема: если переменная типа осталась *только* в ограничениях, тип неоднозначен. Какой тип у `read s` в `show (read s)`?
- Для числовых классов есть правило умолчания (defaulting): пробуются `Integer`, затем `Double`. Отсюда предупреждение `Defaulting the type variable 'a0' to type 'Integer' in the following constraints` и то, что `2^64` в GHCi — `Integer`, а не `Int`.

## Неоднозначные типы и умолчания

- Проблема: если переменная типа осталась *только* в ограничениях, тип неоднозначен. Какой тип у `read s` в `show (read s)`?
- Для числовых классов есть правило умолчания (defaulting): пробуются `Integer`, затем `Double`. Отсюда предупреждение `Defaulting the type variable 'a0' to type 'Integer' in the following constraints` и то, что `2^64` в GHCi — `Integer`, а не `Int`.
- В GHCi действуют расширенные правила умолчания (`ExtendedDefaultRules`): например, для `print []` тип элементов выбирается как `()`, а в файле это ошибка `Ambiguous type variable`.

## Алгоритм $\mathcal{M}$ : сверху вниз

- Алгоритм  $\mathcal{M}$  (Lee, Yi, 1998) идёт сверху вниз:  
 $\mathcal{M}(\Gamma; e; \rho)$  получает ожидаемый тип  $\rho$  и передаёт его подвыражениям; унификация выполняется для переменных и лямбд.

## Алгоритм $\mathcal{M}$ : сверху вниз

- Алгоритм  $\mathcal{M}$  (Lee, Yi, 1998) идёт сверху вниз:  $\mathcal{M}(\Gamma; e; \rho)$  получает ожидаемый тип  $\rho$  и передаёт его подвыражениям; унификация выполняется для переменных и лямбд.
- Переменная  $x$ : унифицируем  $\rho$  с типом  $x$  из  $\Gamma$  (после замены переменных под  $\forall$  на свежие).
- Применение  $e_1 e_2$ : свежая  $\beta$ ;  $\mathcal{M}(\Gamma; e_1; \beta \rightarrow \rho)$ , затем  $\mathcal{M}(\Gamma; e_2; \beta)$ .
- Лямбда  $\lambda x. e_1$ : свежие  $\alpha, \beta$ ; унифицируем  $\rho = \alpha \rightarrow \beta$ ;  $\mathcal{M}(\Gamma, x : \alpha; e_1; \beta)$ .
- **let**  $x = e_1$  **in**  $e_2$ : свежая  $\beta$ ;  $\mathcal{M}(\Gamma; e_1; \beta)$ ; обобщаем  $\phi(\beta)$  как в  $\mathcal{J}$ ;  $\mathcal{M}(\Gamma, x : \sigma; e_2; \rho)$ .
- Для всего выражения:  $\mathcal{M}(\Gamma; e; \alpha)$  со свежей  $\alpha$ ; ответ —  $\phi(\alpha)$ .

## Алгоритм $\mathcal{M}$ : пример и следствия

- `not id` с ожидаемым  $\rho$ : для `not` унифицируем  $\text{Bool} \rightarrow \text{Bool} = \beta \rightarrow \rho$ , получаем  $\beta = \text{Bool}$ ; для `id` унифицируем  $\alpha_1 \rightarrow \alpha_1 = \text{Bool}$  — ошибка на `id`.

## Алгоритм $\mathcal{M}$ : пример и следствия

- `not id` с ожидаемым  $\rho$ : для `not` унифицируем  $\text{Bool} \rightarrow \text{Bool} = \beta \rightarrow \rho$ , получаем  $\beta = \text{Bool}$ ; для `id` унифицируем  $\alpha_1 \rightarrow \alpha_1 = \text{Bool}$  — ошибка на `id`.
- Ошибка обнаруживается раньше, чем в  $\mathcal{J}$  (доказано в статье); здесь — при проверке `id`.
- Так же устроены сообщения GHC: *expected type* — тип, переданный сверху, *actual type* — выведенный для подвыражения.

## Алгоритм $\mathcal{M}$ : пример и следствия

- **not** **id** с ожидаемым  $\rho$ : для **not** унифицируем  $\text{Bool} \rightarrow \text{Bool} = \beta \rightarrow \rho$ , получаем  $\beta = \text{Bool}$ ; для **id** унифицируем  $\alpha_1 \rightarrow \alpha_1 = \text{Bool}$  — ошибка на **id**.
- Ошибка обнаруживается раньше, чем в  $\mathcal{J}$  (доказано в статье); здесь — при проверке **id**.
- Так же устроены сообщения GHC: *expected type* — тип, переданный сверху, *actual type* — выведенный для подвыражения.
- Типы и множество принимаемых программ те же, что у  $\mathcal{J}$  и  $\mathcal{W}$ .
- «Ожидаемый тип как вход» — начало двунаправленного вывода типов (см. дополнительное чтение).

## Дополнительное чтение

- Более известна альтернатива  $\mathcal{J}$ , избегающая «глобальной»  $\phi$  — алгоритм  $\mathcal{W}$ . Пример реализации со ссылками на дополнительные материалы: [Algorithm W Step by Step](#); оба алгоритма с union-find коротко разобраны в [Damas-Hindley-Milner inference two ways](#).
- Глава [Hindley-Milner Inference](#) в незаконченной книге [Write You a Haskell](#)
- Реализация системы типов всего (!) стандарта Haskell 98 с выводом, основанная на  $\mathcal{J}$ , есть в [Typing Haskell in Haskell](#).
- Курс лекций В. Н. Брагилевского "Вывод типов от Хиндли — Милнера до GHC 8.8" (есть видео)

## Дополнительное чтение: современный взгляд

- F. Pottier, A modern eye on ML type inference (2005): вывод типов как решение ограничений; уровни для быстрого обобщения.
- O. Kiselyov, How OCaml type checker works: уровни в реальном компиляторе.
- OutsideIn(X) (2011): вывод в GHC (ограничения и импликации); когда и почему ограничивается обобщение локальных `let`.
- J. Dunfield, N. Krishnaswami, Bidirectional Typing (2021): когда без аннотаций не обойтись (полиморфизм высших рангов).